
Group Relative Policy Optimization for Control

Jannik Busse Guldmand
MSc Data Science (ICT Systems)
Department of Mathematics and Computer Science
University of Southern Denmark
jagul24@student.sdu.dk

DM887 Reinforcement Learning
Term Project: GRPO for Control
Spring 2026
Supervisor: Professor Melih Kandemir
https://github.com/guldmand/DM887_Project

Abstract

This report studies Group Relative Policy Optimization (GRPO) for control. The full report will present a GRPO variant for continuous and discrete control tasks and evaluate it against PPO, SAC, and TD3 on Car Racing, cartpole-swingup-v0, and acrobot-swingup-v0. The midway version focuses on the required interim components: related work, MDP notation, and baseline results for PPO, SAC, and TD3.

1 Introduction

Reinforcement learning for control aims to learn policies through interaction with dynamic environments. In this project, the long-term objective is to investigate whether Group Relative Policy Optimization (GRPO), originally introduced in the context of language-model reinforcement learning and mathematical reasoning, can be adapted to control-oriented benchmark environments [Shao et al., 2024]. Such an adaptation is non-trivial because control tasks require sequential credit assignment, stable learning from trajectories, and evaluation across environments, random seeds, and training budgets.

The final project will compare a GRPO-based control method against PPO, SAC, and TD3. These algorithms represent widely used baselines in deep reinforcement learning: PPO is an on-policy policy-gradient method, SAC is an off-policy maximum-entropy actor-critic method, and TD3 is an off-policy deterministic actor-critic method for continuous control [Schulman et al., 2017, Haarnoja et al., 2018, Fujimoto et al., 2018]. In this midway report, however, the GRPO-control method has not yet been implemented. The contribution is therefore to establish the formal notation, implementation pipeline, and baseline result matrix required for the final comparison.

The midway implementation completes PPO, SAC, and TD3 baselines on three environments: `cartpole_swingup`, `acrobot_swingup`, and `car_racing_continuous`. The two vector-control environments are run through ObjectRL, while CarRacing is handled by project-side CNN implementations because the image-observation setup does not fit directly into the standard vector-observation workflow [Baykal et al., 2025, Towers et al., 2024]. The completed baseline matrix contains three algorithms, three environments, and five seeds, corresponding to 45 runs and 900 evaluation rows.

The results should be interpreted as an interim foundation rather than final performance claims. No hyperparameter optimization was performed, and the training budgets were chosen to validate the pipeline rather than to establish converged performance. The main outcome of this report is therefore

a reproducible experimental foundation: the MDP notation is fixed, the baseline matrix is complete, the result files are validated, and learning curves can be generated from the stored evaluation data.

The remainder of the report is organized as follows. Section 3 introduces the MDP notation and experimental protocol. Section 5 presents the completed midway baseline experiments. The final report will extend the related work, theory, and conclusion sections when the GRPO-control method has been formalized and evaluated.

2 Related Work

This section positions the project relative to the baselines used in the interim experiments and the GRPO motivation for the final project stage. The aim is not to provide a broad survey of reinforcement learning, but to justify the comparison against PPO, SAC, and TD3 and to identify the gap addressed by a later GRPO-control variant. These baselines represent three central families in deep reinforcement learning for control: on-policy policy optimization, off-policy stochastic actor-critic learning, and deterministic actor-critic learning for continuous action spaces.

PPO is the natural on-policy baseline for this project. It optimizes a policy using sampled trajectories and stabilizes policy-gradient updates through a clipped surrogate objective, limiting how far the updated policy can move from the previous policy in a single update [Schulman et al., 2017]. PPO is also closely connected to advantage estimation. Generalized Advantage Estimation provides a practical bias-variance trade-off for estimating the advantage signals used in policy updates [Schulman et al., 2015]. In this report, PPO therefore serves both as an established control baseline and as the closest reference point for the later GRPO discussion.

SAC and TD3 complement PPO by representing off-policy actor-critic approaches. SAC uses a maximum-entropy objective, optimizing both expected return and policy entropy [Haarnoja et al., 2018]. This makes SAC a relevant stochastic off-policy baseline, especially in control tasks where exploration and reuse of past experience are important. TD3 instead follows the deterministic actor-critic line of work developed through deterministic policy gradients and DDPG [Silver et al., 2014, Lillicrap et al., 2015]. It addresses overestimation and function-approximation error through mechanisms such as twin critics, delayed policy updates, and target policy smoothing [Fujimoto et al., 2018]. Together, PPO, SAC, and TD3 provide a baseline set that covers several important design choices in deep reinforcement learning.

GRPO motivates the final project direction, but it is not evaluated as a control method in this interim report. In DeepSeekMath, GRPO is introduced as a PPO-related reinforcement learning method for mathematical reasoning in language models, where the policy update signal is computed relative to groups of sampled outputs [Shao et al., 2024]. This idea is relevant because it suggests an alternative to conventional value-based advantage estimation. However, transferring this idea to online control is not direct. Control tasks involve sequential environment interaction, temporal credit assignment, continuous or structured action spaces, and evaluation across random seeds and training budgets. It therefore remains open whether GRPO-style group-relative updates can be formulated and evaluated meaningfully for control benchmarks.

The project uses ObjectRL and Gymnasium as part of a reproducible experimental framework. ObjectRL provides the baseline implementations used for the vector-control PPO, SAC, and TD3 experiments, while Gymnasium-style interfaces support standardized interaction with environments [Baykal et al., 2025, Towers et al., 2024]. CarRacing is handled through project-side CNN agents because its image observations do not fit directly into the vector-observation workflow used for the ObjectRL-based runs. This tooling choice is not an algorithmic contribution, but it explains how the baseline experiments are organized and validated consistently.

Overall, the related work places the project between established control baselines and recent GRPO-motivated policy optimization from language-model reasoning. The interim report does not answer whether GRPO is better than PPO, SAC, or TD3 for control. Instead, it establishes the necessary comparison basis: a complete and validated PPO/SAC/TD3 baseline matrix on three environments with five seeds. The final project stage will build on this foundation by formulating, implementing, and evaluating a GRPO-control variant under the same experimental structure.

3 Methodology

This section fixes the notation and experimental protocol used in the midway study. The goal is not to present the final GRPO-control method, but to define the Markov decision process (MDP) setting, the baseline comparison setup, and the evaluation metric used for the PPO, SAC, and TD3 experiments.

3.1 MDP Formulation

The control tasks are modelled as Markov decision processes. An MDP is denoted by

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0),$$

where $\mathcal{S}$ is the state or observation space, $\mathcal{A}$ is the action space, $P(s' | s, a)$ is the transition kernel, $r(s, a)$ is the reward function, $\gamma \in [0, 1)$ is the discount factor, and ρ_0 is the initial-state distribution. At time step t , the agent observes s_t , selects an action a_t , receives reward r_{t+1} , and transitions to s_{t+1} .

A stochastic policy is written as $\pi_\theta(a | s)$, while a deterministic policy is written as $\mu_\theta(s)$. PPO and SAC use stochastic policies in this report, whereas TD3 uses a deterministic actor. A policy induces trajectories

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots),$$

and the discounted return from time t is

$$G_t^\gamma = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The value function, action-value function, and advantage function are defined as

$$V^\pi(s) = \mathbb{E}_\pi[G_t^\gamma | S_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t^\gamma | S_t = s, A_t = a],$$

and

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

This notation is sufficient for the midway report, where the focus is on baseline algorithms and experimental validation. The final report will extend this notation when the GRPO-control update is formalized.

3.2 Baseline Algorithms

The midway experiments use PPO, SAC, and TD3 as baselines. PPO is an on-policy policy-gradient method based on clipped policy updates, SAC is an off-policy maximum-entropy actor-critic method, and TD3 is an off-policy deterministic actor-critic method designed to reduce overestimation errors in continuous control [Schulman et al., 2017, Haarnoja et al., 2018, Fujimoto et al., 2018]. In this report, these algorithms are not the proposed contribution; they establish the comparison surface required for the later GRPO-control evaluation.

The baseline matrix covers three environments and five random seeds. The vector-control environments are `cartpole_swingup` and `acrobot_swingup`. The image-based environment is `car_racing_continuous`. The resulting midway matrix is

$$3 \text{ algorithms} \times 3 \text{ environments} \times 5 \text{ seeds} = 45 \text{ runs.}$$

No hyperparameter optimization was performed, so the results should be interpreted as preliminary baseline behaviour under a fixed experimental setup.

3.3 Implementation Setup

The implementation uses two paths because the environments differ in observation type. The vector-control environments are executed through ObjectRL [Baykal et al., 2025]. A project-side environment bridge exposes the environments through a Gymnasium-style interface and converts observations to one-dimensional `float32` vectors compatible with the ObjectRL baseline workflow.

The CarRacing environment uses image observations and is therefore handled by a project-side CNN implementation. The observations are converted from height–width–channel to channel–height–width format,

$$(96, 96, 3) \rightarrow (3, 96, 96),$$

and scaled to floating point values before being passed to convolutional policy and critic networks. The CarRacing runs were executed on Google Colab with CUDA and copied back into the local repository, where they are processed in the same result pipeline as the vector-control runs.

3.4 Evaluation Protocol

The reported metric is undiscounted evaluation episode return. This differs from the discounted return used in the RL formulation above. For an evaluation episode of horizon H , the reported return is

$$G^{\text{eval}}(\tau) = \sum_{t=0}^{H-1} r_{t+1}.$$

Learning curves therefore show training steps before evaluation on the horizontal axis and observed undiscounted evaluation return on the vertical axis.

The vector-control environments are evaluated over a midway budget of 20,000 training steps, while CarRacing is evaluated over 10,000 training steps. The final notebook validates that all 45 algorithm–environment–seed combinations are present and that the result set contains 900 evaluation rows. These validated results are then used for the aggregated tables and learning curves in Section 5.

3.5 Role in the Final GRPO Project

At the midway stage, the methodology establishes the notation, baseline matrix, implementation setup, and evaluation protocol. The final project stage will add the GRPO-control method and evaluate it under the same environment set, seed structure, and evaluation metric. This makes the current methodology a reproducible foundation for the later comparison rather than the final algorithmic contribution itself.

4 Theory

5 Experiments

The midway experiments evaluate the completed PPO, SAC, and TD3 baseline matrix. The purpose is not to evaluate the final GRPO-control method, which is left for the final project stage, but to validate the experimental pipeline and establish a reproducible baseline foundation for the later GRPO comparison.

5.1 Baseline Matrix and Result Validation

The experiment matrix consists of three algorithms, three environments, and five random seeds. The algorithms are PPO, SAC, and TD3. The environments are `cartpole_swingup`, `acrobot_swingup`, and `car_racing_continuous`. The seeds are 0, 1, 2, 3, 4, giving

$$3 \text{ algorithms} \times 3 \text{ environments} \times 5 \text{ seeds} = 45 \text{ runs}.$$

The final report-facing notebook validates that all 45 expected algorithm–environment–seed combinations are present. It also confirms that the result set contains 900 evaluation rows, that all three algorithms and all three environments are represented, and that all evaluation rows have status completed. The reported figures and summaries are therefore based on a complete midway baseline matrix rather than a partial subset of experiments.

5.2 Baseline Learning Curves

Figure 1 shows the midway learning curves for the two vector-control environments. The horizontal axis gives the number of training steps before evaluation, and the vertical axis gives the undiscounted

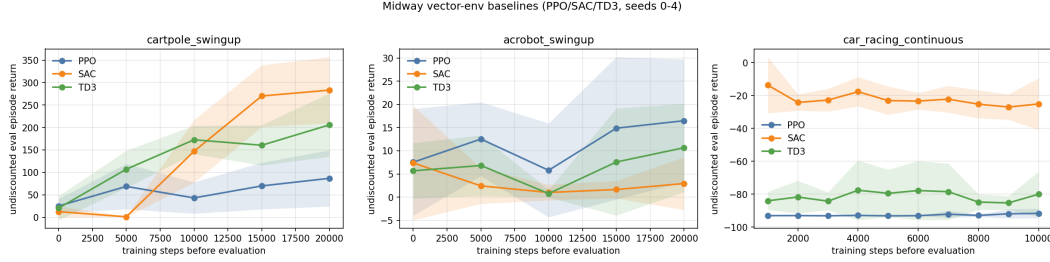


Figure 1: Midway PPO, SAC, and TD3 baseline learning curves for the vector-control environments.

evaluation episode return. The vector-control experiments use a midway budget of 20,000 training steps. The CarRacing experiments use a shorter budget of 10,000 training steps and are included in the same validated result pipeline.

The clearest learning signal appears in `cartpole_swingup`. At the midway budget, SAC obtains the highest mean final return, followed by TD3. Both methods improve from the first to the final evaluation point, while PPO ranks lower in the final evaluation summary.

The results for `acrobot_swingup` are more modest. PPO obtains the highest mean final return at the midway budget, followed by TD3 and SAC. PPO and TD3 show positive development across the run, while SAC does not show the same improvement under the present short setup. This suggests that the environment is more difficult under the selected midway budget and may require longer training or tuning before stronger conclusions can be drawn.

For `car_racing_continuous`, the final returns remain negative for all three algorithms, which is expected under a short CNN-based midway budget. SAC obtains the best mean final return among the CarRacing baselines, followed by TD3 and PPO. The main midway result for CarRacing is therefore not high performance, but that the project-side CNN pipeline runs successfully for PPO, SAC, and TD3 and that the image-based environment is included in the same validated baseline matrix as the vector-control environments.

5.3 Interim Interpretation and Limitations

Overall, the experiments show that the baseline pipeline is functional and complete. The project can run PPO, SAC, and TD3 on all three environments, collect results for five seeds, validate the expected result matrix, and generate learning curves from the stored evaluation data. This establishes the comparison basis required for the final GRPO-control evaluation.

The results should be interpreted cautiously. The training budgets are short, and no hyperparameter optimization was performed. Differences between PPO, SAC, and TD3 may therefore reflect both algorithmic differences and the selected baseline configurations. The results also show variation across seeds and evaluation points, which is expected in reinforcement learning due to initialization, exploration, and environment interaction.

The midway experiments do not answer the full research question of the project. They do not evaluate the final GRPO-control method and should not be used to claim general algorithmic superiority. Their role is to establish a reproducible baseline foundation. In the final stage, the GRPO-control method should be evaluated on the same environments, with the same seed structure and evaluation metric, so that the comparison with PPO, SAC, and TD3 remains consistent.

6 Conclusion

Disclaimer: AI Usage

AI tools were used extensively for code development and report writing support. The main tools were Claude Code, GitHub Copilot CLI, OpenAI Codex, OpenAI ChatGPT 5.5, and the GitHub Copilot Chat extension in Visual Studio Code, especially the `/plan` and `/agent` workflows.

AI was used for planning, debugging, notebook refinement, report-section drafting, LaTeX review, academic wording, and consistency checks between the assignment, code, notebook, and report. Examples of prompts and agent instructions are included in the `prompts/` folder.

All final implementation choices, experimental design decisions, result validation, interpretation of outputs, and report content remain my responsibility. AI tools were not used to fabricate experimental results. The reported results are based on project-generated CSV files and figures validated in the final midway notebook.

References

- Gulcin Baykal, Abdullah Akg"ul, Manuel Haussmann, Bahareh Tasdighi, Nicklas Werge, Yi-Shan Wu, and Melih Kandemir. Objectrl: An object-oriented reinforcement learning codebase. *arXiv preprint arXiv:2507.03487*, 2025. URL <https://arxiv.org/abs/2507.03487>.
- Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, 2018. URL <https://arxiv.org/abs/1802.09477>.
- Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018. URL <https://arxiv.org/abs/1812.05905>.
- Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015. URL <https://arxiv.org/abs/1509.02971>.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015. URL <https://arxiv.org/abs/1506.02438>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International Conference on Machine Learning*, pages 387–395, 2014. URL <https://proceedings.mlr.press/v32/silver14.html>.
- Mark Towers, Ariel Kwiatkowski, Jordan K. Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL <https://arxiv.org/abs/2407.17032>.

A Proofs